AMBA APB Master-Slave RTL Design
Verification Plan

Internship Project — RTL Design & Verification

Title	AMBA APB Master-Slave RTL Design Verification Plan
File	apb_top.v (top), apb_master.v, apb_interconnect.v, apb_slave_ram.v, apb_slave_reg.v, apb_slave_rom.v
Author	Raghav Bansal
Created On	2026-06-30
Tool	Icarus Verilog / GTKWave
Protocol	AMBA APB (Advanced Peripheral Bus) — APB3 compliant
Parameters	ADDR_WIDTH=8, DATA_WIDTH=8, MEM_DEPTH=8, N_RAM=4, N_ROM=2, N_REG=0

1. DUT Overview

This document describes the verification strategy for an AMBA APB (Advanced Peripheral Bus) Master-Slave RTL system implemented in Verilog. The design consists of five synthesizable RTL modules and a self-checking testbench validated on EDA Playground using Icarus Verilog.

1.1 Module Summary

- apb_master.v — 4-state FSM (IDLE / SETUP / ACCESS / DONE); drives PSEL, PENABLE, PWRITE, PADDR, PWDATA; captures PRDATA, PSLVERR; exposes cmd_* interface to testbench
- apb_interconnect.v — Address decoder (upper nibble of PADDR); fan-out of control signals to all slaves; response MUX back to master; error terminator for unmapped addresses
- apb_slave_ram.v — 8-deep byte-wide RAM; FSM with configurable N=4 wait states; PSLVERR on out-of-range address
- apb_slave_reg.v — 8-entry register file; N=0 wait states (PREADY same cycle as PENABLE); addr always valid when selected by interconnect
- apb_slave_rom.v — 8-entry read-only ROM; N=2 wait states; pre-loaded pattern (i×0x11); write attempts rejected with PSLVERR
- apb_top.v — Top-level structural wrapper; instantiates all 5 sub-modules and wires them together

1.2 Address Map

- 0x00 – 0x0F → Slave 0: RAM (ADDR[7:4] == 0x0)
- 0x10 – 0x1F → Slave 1: Register File (ADDR[7:4] == 0x1)
- 0x20 – 0x2F → Slave 2: ROM (ADDR[7:4] == 0x2)
- 0x30 – 0xFF → Unmapped — Interconnect error terminator fires

1.3 Key RTL Fixes (Bugs Found During Verification)

- Bug 1 — Master ACCESS state: PSEL was not explicitly driven high; fixed by adding PSEL<=1 at entry of ACCESS always block
- Bug 2 — Interconnect response MUX: missing !m_PSEL guard caused spurious PREADY=1 on idle bus; fixed with top-level idle check
- Bug 3 — ROM Slave: wait_counter increment did not explicitly stay in ACCESS state; fixed with state<=ACCESS in wait branch
- Bug 4 — ROM Slave: write path silently wrote to rom[]; fixed by adding PSLVERR=1 and removing write to rom[]

2. Interface Overview

Signal	Direction	Driver	Description
PCLK	Input	Master	Clock signal for APB operations
PRESETn	Input	Master	Active-low synchronous reset
PSEL	Output	Master	Peripheral select — asserted in SETUP phase

PENABLE	Output	Master	Enable — asserted in ACCESS phase
PWRITE	Output	Master	1=Write, 0=Read
PADDR[7:0]	Output	Master	Address bus (8-bit)
PWDATA[7:0]	Output	Master	Write data bus (8-bit)
PRDATA[7:0]	Input	Slave	Read data returned by selected slave
PREADY	Input	Slave	Slave ready; extends transfer if low
PSLVERR	Input	Slave	Slave error flag; valid when PREADY=1
cmd_addr[7:0]	Input	TB	Address driven from testbench command interface
cmd_wdata[7:0]	Input	TB	Write data from testbench
cmd_rw	Input	TB	1=Write, 0=Read (testbench command)
cmd_start	Input	TB	Pulse high to initiate a transaction
cmd_ready	Output	Master	HIGH when master can accept a new command
cmd_done	Output	Master	Single-cycle pulse when transaction completes
cmd_rdata[7:0]	Output	Master	Data captured on read completion
cmd_error	Output	Master	PSLVERR captured at transaction completion

3. Verification Objectives

- Verify correct APB protocol sequencing: IDLE → SETUP → ACCESS → DONE state transitions in the master FSM
- Validate all three slave types (RAM, Register File, ROM) for correct read, write, and error behavior
- Confirm address decode in the interconnect correctly selects s0/s1/s2 based on PADDR upper nibble
- Test error handling: out-of-range RAM address, ROM write attempt, unmapped address (error terminator)
- Verify wait-state handling: N=4 for RAM, N=2 for ROM, N=0 for Register File
- Validate reset behavior: synchronous deassertion and proper bus cleanup
- Confirm back-to-back transfer support (same slave and cross-slave)
- Run protocol checkers via apb_protocol_checker on live simulation signals
- Achieve 100% functional, FSM state, and code coverage
- Validate random stimulus handling through self-checking scoreboard

4. Verification Environment

4.1 Testbench Architecture

- tb_top.v — Top-level testbench module; instantiates apb_top (DUT), apb_protocol_checker, and drives all test scenarios via the cmd_* interface
- tb_tasks.v — Reusable APB transaction tasks (apb_write, apb_read, apb_transaction, check_result, print_summary); software scoreboard (expected_ram/reg/rom arrays); test statistics (pass_count, fail_count, test_num)
- test_scenarios.v — Individual test tasks (test1_ram_basic through test8_random); each encapsulates stimulus generation and self-checking

- apb_protocol_checker.v — Passive monitor; samples APB bus on every posedge PCLK; flags 5 protocol violations with $display messages and error flags

4.2 Clock and Reset

- PCLK: 100 MHz, generated as always #5 PCLK = ~PCLK in tb_top
- PRESETn: Driven exclusively from tb_top to avoid multi-driver issues; driven low for 25 ns at startup
- trigger_reset() task defined in tb_top (not in included files) to avoid hierarchical reference issues

4.3 Scoreboard

- expected_ram[0:7], expected_reg[0:7], expected_rom[0:7] — software memory models initialized in init_scoreboard()
- ROM values pre-loaded with i×0x11 pattern matching the RTL initial block
- check_result() task increments pass_count/fail_count and logs [PASS]/[FAIL] with test number and message
- print_summary() prints final pass rate as a percentage at end of simulation

5. Functional Testcases

TC No.	Feature	Testcase Name	Description	Expected Outcome	Status
TC00	General	**APB Master-Slave General Test**	Inject write+read to all three slaves in sequence; verify data round-trip	All reads return expected data; cmd_error==0 for valid addresses	—
TC01	Reset	**Reset Assertion Check**	Assert PRESETn=0; verify PSEL, PENABLE, PWRITE, PADDR, PWDATA all deassert to 0	All APB outputs=0; Master enters IDLE	—
TC02	Reset	**Post-Reset cmd_ready**	After PRESETn deasserts, cmd_ready must go high within 1 clock cycle	cmd_ready==1 on first posedge after reset deassertion	—
TC03	RAM Slave	**RAM Basic Write**	Write 0xAA to addr 0x02; N=4 wait states; verify cmd_done and no error	cmd_error==0; internal mem[2]==0xAA	—
TC04	RAM Slave	**RAM Basic Read**	Read RAM[0x02] after TC03 write; expect cmd_rdata=0xAA	rdata==0xAA AND cmd_error==0	—
TC05	RAM Slave	**RAM Boundary Low (0x00)**	Write 0x10 to addr 0x00; read back	rdata==0x10 AND error==0	—
TC06	RAM Slave	**RAM Boundary High (0x07)**	Write 0xFF to addr 0x07; read back	rdata==0xFF AND error==0	—
TC07	RAM Slave	**RAM Out-of-Range (0x08)**	Write to addr 0x08 (beyond MEM_DEPTH); PSLVERR must assert	cmd_error==1; memory not corrupted	—
TC08	REG Slave	**Register File Basic Write**	Write 0x55 to REG[0x12]; zero wait state (N=0); PREADY next cycle	cmd_done after 2 clocks; cmd_error==0	—
TC09	REG Slave	**Register File Basic Read**	Read REG[0x12]; verify 0x55	rdata==0x55 AND error==0	—

TC10	REG Slave	**Register Back-to-Back Writes**	Write to REG[0x10], REG[0x11], REG[0x13] with no idle between	All 3 cmd_done pulses; no errors	—
TC11	ROM Slave	**ROM Valid Read Pattern**	Read ROM[0x20]=0x00, ROM[0x21]=0x11, ROM[0x22]=0x22, ROM[0x27]=0x77	All reads match pre-loaded i*0x11 pattern; error==0	—
TC12	ROM Slave	**ROM Write Rejected**	Write 0xFF to ROM[0x22]; ROM slave asserts PSLVERR=1	cmd_error==1; ROM data unchanged	—
TC13	ROM Slave	**ROM Integrity After Write Attempt**	Read ROM[0x22] after TC12 attempt; must still be 0x22	rdata==0x22; error==0	—
TC14	Interconnect	**Unmapped Address Write (0x35)**	Write to unmapped region; Interconnect error terminator fires PREADY=1, PSLVERR=1	cmd_error==1; Master not hanging	—
TC15	Interconnect	**Unmapped Address Read (0x3F)**	Read from unmapped region; same error terminator expected	cmd_error==1; cmd_rdata=0x00	—
TC16	Master FSM	**State Progression Check**	Verify PSEL low in IDLE, PSEL high in SETUP, PENABLE high in ACCESS	Waveform matches APB spec timing diagram	—
TC17	Master FSM	**Cross-Slave Back-to-Back Transfer**	Write RAM[0x03]=0xCC then REG[0x14]=0xDD; read both back	Both values correct; no errors	—
TC18	Reset Mid-Tx	**Reset During ACCESS Phase**	Start RAM write (N=4), assert PRESETn after cycle 1, verify RAM[0x05]==0x00	RAM zeroed; Master back to IDLE post-deassertion	—
TC19	Protocol	**PSLVERR Timing Checker**	Verify via protocol checker that PSLVERR never asserts without simultaneous PREADY	error_pslverr_without_pready==0 throughout simulation	—
TC20	Protocol	**Signal Stability During Wait**	During RAM ACCESS wait (PREADY=0) check PADDR, PWRITE do not change	No protocol checker errors in output	—
TC21	Random	**Random Transactions (20 cycles)**	Random addr 0–47, data, rw; scoreboard tracks expected values	Scoreboard pass_count matches all valid checks; fail_count==0	—

6. Checkers

Protocol checkers are implemented as synthesizable Verilog checks inside apb_protocol_checker.v. Each checker samples APB signals on every posedge PCLK using previous-cycle registers and flags violations with both a $display message and a registered error flag output. This allows checkers to be verified both by visual scan of $display output and by waveform inspection of the error flag signals.

Checker ID	Module	Property Name	Description	Expected Outcome
C01	apb_protocol_checker	**penable_requires_psel**	PENABLE may only assert when PSEL was high the previous cycle	error_PENABLE_without_PSEL fires; $display printed
C02	apb_protocol_checker	**addr_stable_during_wait**	PADDR must remain stable during ACCESS wait (PSEL=1,PENABLE=1,PREADY=0)	error_addr_change_during_access fires
C03	apb_protocol_checker	**pwrite_stable_during_wait**	PWRITE must remain stable throughout ACCESS phase wait states	error_pwrite_change_during_access fires
C04	apb_protocol_checker	**pslverr_valid_with_pready**	PSLVERR is only valid when PREADY is simultaneously asserted	error_pslverr_without_pready fires
C05	apb_protocol_checker	**prdata_stable_during_wait**	PRDATA must not change during ACCESS wait on a read transaction	error_prdata_change_during_access fires
C06	apb_slave_ram	**ram_addr_valid**	Write/read succeeds only for mem_index < MEM_DEPTH (8)	PSLVERR=1; memory not written for OOB addr
C07	apb_slave_rom	**rom_write_rejected**	Any write attempt to ROM returns PSLVERR=1; rom[] unmodified	PSLVERR=1; rom data unchanged after attempt
C08	apb_interconnect	**error_terminator**	Unmapped address with PSEL=1 but no sN_PSEL forces PREADY=1,PSLVERR=1	Master gets PREADY=1,PSLVERR=1; no bus hang
C09	apb_master	**cmd_done_pulse**	cmd_done must be exactly 1 cycle wide (single-cycle pulse)	cmd_done high for 1 clock only; default=0 all other cycles
C10	apb_master	**psel_stable_in_access**	PSEL must stay asserted throughout ACCESS phase until PREADY	PSEL==1 while PENABLE==1 (fixed RTL bug)

7. Coverage Plan

Type	Coverage Point	Bins / Items	Target %
Functional	**Master FSM States**	IDLE, SETUP, ACCESS, DONE	**100%**
Functional	**Master FSM Transitions**	IDLE→SETUP, SETUP→ACCESS, ACCESS→DONE/SETUP	**100%**
Functional	**Slave Selection**	s0_PSEL (RAM), s1_PSEL (REG), s2_PSEL (ROM)	**100%**
Functional	**Error Conditions**	OOB addr, ROM write, unmapped addr	**100%**
Functional	**Wait State Variants**	N=0 (REG), N=2 (ROM), N=4 (RAM)	**100%**
Functional	**Reset Paths**	Reset in IDLE, Reset in ACCESS	**100%**

Code	**Line Coverage**	All RTL lines across 5 modules	**100%**
Code	**Branch Coverage**	All if/else and case branches in FSMs	**95%+**
Code	**Toggle Coverage**	PSEL,PENABLE,PWRITE,PADDR[7:0],PWDATA[7:0]	**90%+**
FSM	**RAM Slave FSM**	IDLE, SETUP, ACCESS (wait counter 0..3)	**100%**
FSM	**REG Slave FSM**	IDLE, SETUP, ACCESS (zero wait)	**100%**
FSM	**ROM Slave FSM**	IDLE, SETUP, ACCESS, write-reject path	**100%**
Protocol	**PENABLE Timing**	Checker: PENABLE only after PSEL	**100%**
Protocol	**PSLVERR Timing**	Checker: PSLVERR only with PREADY	**100%**
Protocol	**Signal Stability**	Checker: PADDR/PWRITE stable during wait	**100%**
Random	**Address Space Coverage**	0x00–0x2F valid; 0x30–0x3F invalid region	**90%+**

8. Simulation Parameters & Notes

Clock Frequency: 100 MHz (PCLK period = 10 ns)

Reset Duration: 25 ns (2.5 clock cycles) at startup

Addr Width: 8-bit

Data Width: 8-bit

MEM_DEPTH: 8 locations per slave

N (RAM wait): 4 clock cycles

N (ROM wait): 2 clock cycles

N (REG wait): 0 (zero wait — immediate PREADY)

ROM Init Pattern: rom[i] = i * 8'h11 (0x00, 0x11, 0x22, ..., 0x77)

VCD Dump: apb_top.vcd — full hierarchy dump enabled

Tool: Icarus Verilog 12.0

Result: All 21+ test cases pass with 100% success rate

9. ARCHITECTURE DESIGN DIAGRAM: